

Module 12 — Productionisation

Production-ready AI

🎯 Learning goals

- Implement **structured tracing** with trace IDs and spans for every request.
- Apply **reliability patterns**: retries with backoff, timeouts, circuit breakers, fallbacks.
- Enforce **cost controls**: token budgets, model tiering, caching.
- Deploy with **environment config**, secrets, containers, and health checks — including a **Google Cloud** path.



Structured tracing

Know exactly what happened on every request

Why tracing matters

- A crew member reports a wrong answer this morning. Which request was it, and what did the model see?
- Every request gets a **trace ID** — one identifier carried through every step it touches.
- Each step is a **span** with timing, status, and metadata.
- Log spans as JSON. In production, send to **Datadog**, **Jaeger**, or **OpenTelemetry**.

TraceContext

```
class TraceContext:
    def __init__(self):
        self.trace_id = str(uuid.uuid4())
        self.spans: list[dict] = []

    def start_span(self, name: str) -> dict:
        span = {
            "span_id": str(uuid.uuid4()),
            "trace_id": self.trace_id,
            "name": name,
            "start_time": time.time(),
        }
        self.spans.append(span)
        return span

    def end_span(self, span, status="ok", metadata=None):
        span["end_time"] = time.time()
        span["duration_ms"] = (span["end_time"] - span["start_time"]) * 1000
        span["status"] = status
        if metadata:
            span["metadata"] = metadata
```

- → Every request gets a **trace_id** — propagate it through every function.
- → Each span captures **timing** and **status** — find the slow step instantly.
- → Log as **JSON** — structured logs are searchable, unstructured logs are not.



Reliability patterns

Dependencies fail. Decide what happens next.

What can go wrong

- **Network blips** — transient connection failures to LLM APIs.
- **Rate limits** — API returns 429 when you hit the quota.
- **Timeouts** — the model takes too long and your server thread is still waiting.
- **Cascading failures** — one slow dependency ties up the resources every other request needs.
- Four answers: **retries**, **timeouts**, **circuit breakers**, **fallbacks**.

Retries with exponential backoff

```
async def retry_with_backoff(fn, max_retries=3, base_delay=1.0):
    for attempt in range(max_retries):
        try:
            return await fn()
        except TransientError:
            if attempt == max_retries - 1:
                raise
            delay = base_delay * (2 ** attempt) + random.uniform(0, 0.5)
            await asyncio.sleep(delay)
```

- → **Exponential backoff**: 1s → 2s → 4s — gives the service time to recover.
- → **Jitter** (random.uniform) spreads out clients that all failed at the same moment.
- → Only retry on **transient errors** (5xx, timeout, rate limit) — never on 4xx.

Circuit breaker

```
class CircuitBreaker:
    def __init__(self, failure_threshold=5, recovery_time=60):
        self.failures = 0
        self.threshold = failure_threshold
        self.recovery_time = recovery_time
        self.state = "closed"  # closed = normal, open = failing

    async def call(self, fn):
        if self.state == "open":
            if time.time() - self.last_failure > self.recovery_time:
                self.state = "half-open"  # allow one test request
            else:
                raise CircuitOpenError("Service unavailable")
        try:
            result = await fn()
            self.failures = 0
            self.state = "closed"
            return result
        except Exception:
            self.failures += 1
            self.last_failure = time.time()
            if self.failures >= self.threshold:
                self.state = "open"
            raise
```

- → **Closed** = normal operation. **Open** = requests rejected locally, none sent.
- → After recovery_time, enters **half-open** — one test request decides.
- → A dead dependency stops occupying connections that healthy requests need.

Without vs with reliability patterns

No protection

No timeout → the thread waits on a call that never returns

Rate limit → immediate retry → rate limited again

Service down → every request waits its full timeout

The crew sees errors and spinners

With patterns

Timeout fires → fallback response in 2 seconds

Backoff + jitter → recovers without a coordinated retry spike

Circuit breaker → reject locally, try the fallback model

The crew gets an answer, sometimes a degraded one

🔄 Fallback chains

- When the primary model fails, fall through the chain rather than returning an error.
- Fallback chain: **primary model** → **cheaper model** → **cached response** → **plain apology**.
- Each rung is a worse answer than the one above it, and all of them beat a 500.
- `asyncio.wait_for(call, timeout=30)` — every external call gets a timeout.



Cost controls

An agent loop has no natural stopping point

Token budget tracking

```
class CostTracker:
    def __init__(self, session_budget=10_000, daily_budget=1_000_000):
        self.session_usage = 0
        self.daily_usage = 0
        self.session_budget = session_budget
        self.daily_budget = daily_budget

    def record(self, prompt_tokens: int, completion_tokens: int):
        total = prompt_tokens + completion_tokens
        self.session_usage += total
        self.daily_usage += total

    def within_budget(self) -> bool:
        return (self.session_usage < self.session_budget
                and self.daily_usage < self.daily_budget)
```

- → Track usage **per session** and **per day**. Reject requests over budget.
- → **Model tiering** is the biggest lever: classification → cheap model, reasoning → expensive.
- → **Caching** similar queries avoids redundant LLM calls entirely.



Cloud production

Map patterns to your provider

Production patterns at a glance

AWS

- **API** — App Runner, ECS Fargate
- **Secrets** — Secrets Manager, SSM
- **Logs / trace** — CloudWatch, X-Ray
- **Managed LLM** — Amazon Bedrock
- **Cost** — AWS Budgets, quotas

Azure

- **API** — Container Apps, App Service
- **Secrets** — Key Vault
- **Logs / trace** — App Insights, Monitor
- **Managed LLM** — Azure OpenAI
- **Cost** — Cost Management, AOAI quotas

GCP

- **API** — Cloud Run, GKE
- **Secrets** — Secret Manager
- **Logs / trace** — Cloud Logging, Trace
- **Managed LLM** — Vertex AI
- **Cost** — Billing budgets, quotas



AWS

Enterprise AI on the most common cloud



When teams choose AWS

AWS is used when an organisation already runs on Amazon infrastructure or needs broad service coverage and compliance programs. For AI production you keep the same patterns from this module — trace IDs, retries, budgets — but wire them to **CloudWatch**, **Secrets Manager**, and optionally **Bedrock** instead of calling OpenAI directly. You deploy your FastAPI image to **App Runner** or **Fargate** and inject secrets at runtime, not in the Docker image.

AWS service mapping

- **Run container** — App Runner (simplest) or ECS Fargate for a FastAPI image.
- **Secrets** — Secrets Manager or SSM Parameter Store for `OPENAI_API_KEY`; never bake into the image.
- **Tracing** — JSON logs with `trace_id` → CloudWatch Logs; optional X-Ray or ADOT OpenTelemetry.
- **Managed LLM** — Amazon Bedrock when policy requires models inside your AWS account.
- **Cost** — AWS Budgets + service quotas; align with in-app `CostTracker` session/daily limits.



Azure

Microsoft cloud + Azure OpenAI

When teams choose Azure

Azure fits Microsoft-centric enterprises: Entra ID, existing subscriptions, and **Azure OpenAI Service** for GPT models behind a private endpoint. Production AI apps use **Container Apps** or **App Service** for the FastAPI container, **Key Vault** for API keys, and **Application Insights** for the same span JSON you log in `TraceContext`. The course code stays portable Python; on Azure you swap the OpenAI client endpoint and deployment name while keeping tracing and cost patterns unchanged.

Azure service mapping

- **Run container** — Azure Container Apps (scale-to-zero) or App Service with Linux containers.
- **Secrets** — Key Vault references injected as env vars into Container Apps / App Service.
- **Tracing** — Application Insights ingests structured logs and dependency spans.
- **Managed LLM** — Azure OpenAI: resource endpoint + deployment name (not raw api.openai.com).
- **Cost** — Cost Management budgets; AOAI TPM/RPM quotas mirror token budget enforcement.



Deploying to Google Cloud

Hosting, data, security, and the model in one project



When teams choose Google Cloud

Google Cloud fits a container-native API: **Cloud Run** runs one Docker image, and **Vertex AI** hosts Gemini and other models in-region. The patterns from this module stay the same. `TraceContext` JSON goes to **Cloud Logging**, keys live in **Secret Manager**, and latency goes to **Cloud Trace**. The Dockerfile from the deployment section maps onto Cloud Run: one service, env config, and `/health` for the probe.

Google Cloud: Hosting

- **Cloud Run** — one container behind an HTTPS URL. Scales to zero, billed per request. Best fit for the FastAPI image in this module.
- **Cloud Run functions** — one function for a small job: an event, a queue message, or a schedule. Not the place for the whole API.
- **GKE** — Kubernetes. Many services, GPUs, custom networking, and sidecars. You run the node pools. Choose it when a Cloud Run limit binds.
- **Compute Engine** — a virtual machine. Use it for a box you log into, or a GPU that is not part of a cluster.

Google Cloud: AI friendly Databases

- **Vertex AI Vector Search** — Google's managed service for indexing and searching vectors.
- **Cloud Storage** — Google's object storage service—broadly equivalent to Amazon S3.
- **Memorystore** — In memory Redis-like key-value store.
- **Firestore** — a NoSQL document database, storing data in collections and documents, with real-time updates and offline support.

Google Cloud: Vector stores

- **RAG Engine** — Managed RAG service, an use Vertex under the hood.
- **Vertex AI Vector Search** — Google's managed service for indexing and searching vectors.
- **pgvector on Cloud SQL or AlloyDB** — adds vector search to PostgreSQL

Google Cloud: LLM services

- **Gemini** — Google models behind a managed API. Prompt in, text out. No GPU to run.
- **Model Garden** — the catalog. Gemini and Gemma, partner models such as Claude and Mistral, and open models such as Llama.
- **Model as a Service** — partner and open models on a managed API. Same call shape as Gemini. You do not deploy them.

Google Cloud: Security

- **Secret Manager** holds API keys and `DATABASE_URL`. Inject them with `--set-secrets`.
- **Artifact Registry** stores the image and can scan it on push. The build never sees the secrets.
- **Cloud Armor** provides DDOS protection, like Cloudflare's WAF.

Google Cloud: Logs, traces, and cost

- **Cloud Logging**. Write JSON to stdout. Cloud Logging collects it.
- **Cloud Trace** shows latency across retrieve, model, and guardrail. Export OpenTelemetry if the spans already exist.
- **Billing budgets** and Vertex quotas sit beside `CostTracker`. The app cap and the project cap are different limits.

Exercises

01 — Structured tracing: TraceContext with nested spans, durations, and one JSON log line per span

02 — Reliability: retry with backoff and jitter, a circuit breaker, and a fallback chain

03 — Cost controls: token budgets, a model-tiering router, and a prompt cache



Capstone exercises

Choose one. Build it production-ready.

Pick your capstone

04 — AI Recipe Finder

Photo your fridge, get recipe suggestions.

RAG + hybrid search + multimodal vision.

Allergen guardrails + semantic caching.

05 — AI Movie Night

Describe your mood, get movie picks.

RAG + reranking + text-to-SQL for data questions.

Charts and structured output.

06 — AI Travel Planner

Describe your dream trip, get a day-by-day itinerary.

Agentic RAG + tool calling + web fallback.

Budget tracking + structured itineraries.

07 — AI Personal Assistant

Friendly companion with calendar, notes, reminders.

Chat + tool calling + MCP + RAG over notes.

Session memory + cost controls.

How the capstones work

- Exercises **04–07** in this module. Each has a **frontend/** (SvelteKit — provided), a **starter/** (your code), and a **solution/**.
- You build the **FastAPI backend** in `starter/` — the frontend connects automatically.
- Apply the patterns from this module: **tracing**, **retries**, **cost tracking**. Several also use Module 13 techniques (hybrid search, caching, guardrails).
- Run with `uvicorn starter.app:app --reload --port 8000` and `cd frontend && pnpm dev`.



Module 12 — Complete

Next: Module 13 — Edge Topics